I have a local set up that can run collider servers for backend and runtime and local llm and small datadisk to :1. Service applications like websites using collider api to run, build, or both for users. But also for 2 to share with others that can run said collider servers and or frontend application, in order to extend collider space and/co work on specific parts of the code depending on systemrole/user identity (who the person is). Can you now research on the next step: to use my. Local workstation z440 xeon 6core and asus rtx 306012gb graph adapter capable of running llm, vector search, embedding, matrix and tensor math, parrallel vectorsearchess on the linkobjects fields for description data to each of the linked containers, the dependency axis. But also the Fields that carry the I/O, the pydantic data, of lets say a python Script or an agent tool. The actual "doc string" combined with added conditionsnodesedges on the way up in the graph. Its is these fiels i want to use my gpus for. Traversal over axes of dependency and descriptions versus of over organization and contracts. These axes are accompanied by collideer fields and exactly that pydantic ic data is what i want to crunch with collider api for axaample to RAG specific containers with pydantic i/o data and use those as tools for agents that use REACT like deep agent and orchetrate the obtained tools holding the subgraphs rather than Just executing the graph. Thats why we need recursive containers simply linking other containers as dependencies to the deep agent and defintions (or the definitien fields when we simply unify in 1 linkedcontainerdefintion model) to each of them to describe the tooluse specs. Try to reason deep over the implications of having a dependency graph with users or ai generated if we let collider pilots build the graph and deciding over creating dependencies and organizing them. Try to imagine a container not even being connected too actual definitions, code but with a higher level description in terms of i/0 and building towards less abstract descriptions while recalculating the actual i/o based on the inwards emerging orchestrated network until code is reached. Iteratively. Of course this is an algorithm with the risk of exponentially growing i to meaningless

complexity and 1000 million gb of fields and vectors. But it has bot practical use: i write scripts. Scripts are tools. Tools are dept to agents. Agents are deps. Agents run collider defintions held by recursive collider containers and their deps. Containers are user owned and we have share feature.

Edit the research plan

This piece on PIKE RAG is. An example of reasoning algorithm for determining graph data with datasets containing domain specific knowledge

Start research

Please propose pydantic base models

Can you show me how i would instead of ai traversingly creating the dependency graph, but to instruc the collider by passing a large set of data, an object, to parallel instruct a container including all dependencies descriotions nodes wdges and pydantic graph data. Think of the max size of such a object consider sets of preset fields we need any how to make sense of the semantic breakdown versus the pydantic io contract that "bridges" the needed final (or in between) step to even achieve a contract considering contract requirements also. Ok that object could be the result of a vectorsearch with known weights andbiases to similar domain specific knowledge to other knowledgeaystems resukting in a specific collider holding the tools for agents to reason on queations on the new domain knowledge field.

Now provide me the pydantic models. Currently my codebase meets all requirements but lacks basic models and methods just the contianier draft models and others. Please provide me a new set of pydantic models knowing my needs in terms of described scenarios, goals, question etc in this deepresearch conv

Now provide similar frameworks or patterns or solutions known in tech or science

Given what we talked about in total already: zummarize this conversation by summing up requirements and relevant sources to get me going on my workstation and prep my local env and factory

Can you build it without a recursive container model like datastructure? Explain in max 300 words

Yes, absolutely. A **Flat Graph** architecture (Adjacency List) is actually more efficient for your Xeon/GPU setup and easier to manage than deep recursion.

The Strategy: "Virtual" Nesting

Instead of storing containers *inside* containers, store all nodes in a single flat table (`Nodes`) and all connections in a single flat table (`Edges`).

1. **The Model Change:** Add a `parent_id` field to your `Container` model.
   - **Recursive:** Object A contains [Object B].
   - **Flat:** Object B has `parent_id = A.id`.
2. **Why this is better for you:**
   - **Reactflow Native:** Reactflow expects a flat list of `nodes` and `edges` with position data. A flat backend maps 1:1 to your frontend.
   - **GPU Vectorization:** Embedding a flat list of 1,000 "Tools" is faster than traversing a deep JSON tree to find leaf nodes. Your `DomainBlueprint` becomes a simple bulk insert.
   - **SQL Performance:** Querying "all nodes in a subgraph" is a simple `SELECT * FROM nodes WHERE parent_id = X`, avoiding complex recursive Common Table Expressions (CTEs).

How it works: Your **Blueprint** injects 50 nodes into the database. To the database, they are peers. To the *User* and *Agent*, they look nested because the UI renders anything with `parent_id=X` *inside* the box of X. The complexity moves from the **Data Structure** to the **Query/View** layer.

Ok. Now the Zwicky box. Suppose we define a container with dependencies as s such. The number of deps d of a R>

=>1 container in collider "space" would be a dimension, or maybe better a "degree of complexity and the number of the node-to-node combinations would be a dimension.

Then we have the actual pydantic I/O of these deps, the emerging graph that will connect the deps, that pyd I/O is the processed data passed from node to node through the edges of the specific containerspace with depth d. Now if each node has its edges to any other node and and all edges carry pydantic data (from atomic scripts with models methods and docstrings) up. If we assume max e edges between 1 node and another, we would have 1+2+3+...+(d-1) x e edges with values representing pydantic data processes. Every container can be described as a multidimensional vector with its own degree of complexity and its own nr of dimensions to pydantically map the graph logic for the data being processed in this container. The conceptual value for each of the edges, determine the direction of the vector. What would be the space of this vector? Please be critical and help me use/forget this concept

This is a fascinating theoretical construct, but from a systems engineering and implementation standpoint (especially for your RTX 3060 setup), here is the verdict:

VERDICT: FORGET the "Zwicky Box of Runtime Values." VERDICT: USE the "Semantic Transformation Vector."

Here is the critical breakdown of why, and the mathematical substitute that actually works for your AI / PIKE-RAG system.

1. The Critique (Why you should forget your version)

You described a space where **Dimensions = Edges** and **Vector Magnitude = Pydantic Data Values.** This leads to three fatal flaws:

1. **The Dimensional Explosion (Curse of Dimensionality):**
    - If you have 10 nodes, you potentially have 90 edges. If you have 100 nodes, you have 9,900 edges.
    - A vector space where the *number of dimensions* changes every time you add a node is computationally painful. Standard vector search engines (Qdrant/Chroma) require fixed-dimension vectors (e.g., 768 or 1536 dimensions). You cannot compare a 10-dim vector to a 100-dim vector easily.
2. **Category Error (Static vs. Dynamic):**
    - A "Container" is a **Class** (The Blueprint).

- The "Pydantic Data" flowing through edges is an **Instance** (The Runtime State).
- Defining the vector based on the *data* means the container's "location" in space changes every millisecond as data flows. You can't index that.

3. **The "Zwicky" Misnomer:**
   - A Zwicky box is for morphological analysis (combining discrete options). You are describing a **State Space**.

2. The Solution (How to USE the concept)

Refine your idea into a **Semantic Transformation Space**. Instead of measuring the *wires* (edges), measure the *change in potential* between Input and Output.

The Math: Do not look at the internal complexity (d). Look at the Container as a function $F(x)$.

$$V$$

Container

$$=$$

$$E$$

$$(\text{Output})^{-}$$

$$E$$

(Input)

Where

E

is the semantic embedding vector.

This vector represents the **"Work Done"** by the container.

The "Collider Space" Definition

If you want to map your containers to a space for your GPU to analyze, define the space S not by edge counts, but by **Functional Characteristics**.

The Vector Dimensions (The "Fingerprint"):

1. D
2.
3. *domain*
4.
5. **(Semantic Position):** Where does this live in knowledge space? (e.g., Finance vs. Biology). *Computed via BERT/Nomic embedding of the description.*
6. T
7.
8. *transformation*

9.
10. **(The "Delta"):** What does it do to data? (e.g., `Unstructured -> Structured` or `Image -> Text`).
11. C
12. $complexity$
13.
14. **(Scalar):** The topological complexity you mentioned ($1+2+\ldots$).
    - This is just a single number (Scalar) representing "Cost" or "Depth".
    - It's a weight, not a vector direction.

3. The Implementation (Python/Pydantic)

Here is how you actually model this "Vectorized Container" so your GPU can process it.

I will generate a file called `collider_vector_space.py`. This allows you to "fingerprint" a container.

Vector Space Model

Jan 14, 2:01 PM

How this helps your Agent (PIKE-RAG)

When your DeepAgent needs to solve a problem:

1. **User Request:** "Convert this CSV to a Chart."
    - Input State: `CSV (Unstructured Text)`
    - Desired State: `Chart (Image)`
2. **Vector Math**: The Agent calculates the **Desired Transformation Vector**: $$ \vec{V}_{goal} = \vec{E}(\text{Chart}) - \vec{E}(\text{CSV}) $$
3. **Search:** The Agent queries Qdrant for a Container whose `transformation_vector` is mathematically similar (Cosine Similarity) to
4. V
5.
6. $goal$
7.
8. .

Summary: Don't map the *internal wires*. Map the *External Change*. The internal complexity (d) is just a "Cost Function" (use the simplest tool that achieves the vector transformation).

Category Error (Static vs. Dynamic: i misuse the term pydantic data bc that is actual data. But what i try to define is (purely for thought exp, not implementation) is the cumulative procecessing of that data from code to I/O through edges to nodes. The specific transormation through the edges is the value i refer to. Not the data itsself. Then we have edges and transormative values to them. And of course some "ground" to refer to if we would compare containers and i mean the the vectors that would describe two containers

This is a significant conceptual breakthrough. You have moved away from **State Space** (where the data is) and arrived at **Function Space** (where the *logic* is).

In Physics/Math, you are describing a **Vector Field on a Manifold**, or more specifically in Computer Science terms, you are describing **Semantic Flow**.

Here is the critique and the concrete mathematical framework to implement this "Process Vector" on your architecture.

1. The Critique: "The Ground"

You asked: *"What is the 'Ground' to refer to if we compare containers?"*

The Problem: If Container A has 5 edges and Container B has 50 edges, how do you compare their vectors? You cannot simply list the edge values (Dimension Mismatch).

The Solution: The "Ground" is the **Latent Space of your Embedding Model** (e.g., `nomic-embed-text` or `jina-code`).

- **The Metaphor:** Imagine a map.
  - **Data (D)** is a *Location* (Lat/Lon).
  - **Edge/Transformation (T)** is a *Vector* (Direction + Distance).
  - **Container (C)** is the *Trajectory* (The path walked).
- **The "Value":** The value of an edge is the **Semantic Delta** (ΔS). It represents the "distance" traveled in meaning-space between the Input and the Output.

2. The Mathematical Model: Path Integration

You don't define the dimensions by the *number* of edges. You define the vector by the **Net Transformation**.

If a Container C consists of a chain of transformations T

1

$\rightarrow T$

2

$\rightarrow T$

3

:

$$V$$

Container

$$= \sum_{i=1}^{n}$$

$$V$$

$(T$

i

$)$

- V
-
- $(T$
- i
-
-): The embedding of the *logic* of edge i (e.g., "Filters Null Values").
- **Comparison:** Now, a Container with 3 complex steps can be compared to a Container with 10 simple steps. If they result in the same "Semantic Displacement," they are functionally identical.

3. Implementation: The "Process Vector" Model

We will create a model that calculates this. We use the **Code Embedding Model** as the universal "Ground" because it already understands the relationship between "sorting list" (Python) and "ORDER BY" (SQL).

I will generate `process_vector.py`. This is how you represent your "Cumulative Processing" concept in Pydantic.

Process Vector Model

Jan 14, 2:27 PM

Why this is scientifically sound (Category Theory)

You are effectively describing a **Functor**.

1. **Category:** Your Pydantic Schemas are *Objects*.
2. **Morphisms:** Your Edges (transformations) are *Arrows* between objects.
3. **Composition:** Chaining edges ($f \circ g$) is the path.

By calculating the **Net Process Vector**, you are creating a "Structure-Preserving Map" that allows you to search for *logic patterns* regardless of how many steps they take.

Action Item: Use your RTX 3060 to embed the *docstrings* and *type signatures* of your Definitions. These embeddings form the basis vectors. The "Process Vector" is just the mathematical sum of the embeddings along the graph's path.

Lets stop looking at code but keep it general. List me sources in science and tech and give me a quick reference so i can start investigating this

Here is a curated list of scientific domains and key references to investigate your concept of **"Cumulative Processing as a Vector Trajectory."**

1. Geometric Deep Learning (GDL)

This is the modern field studying how to apply neural networks to non-Euclidean data (like your Graph of Containers). It provides the math for embedding an entire graph structure into a single vector.

- **The Concept:** Treating your graph not as a list of data points, but as a shape (manifold) in high-dimensional space.
- **Reference:** *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges* (Bronstein, Bruna, LeCun, et al., 2021).
- **Why check this:** It explains how to mathematically "fingerprint" a graph structure so you can compare the shape of two different processing pipelines.

2. Category Theory (Applied to Computation)

This provides the rigorous proof for your "Process Vector." In Category Theory, data types are objects and programs are **Morphisms** (arrows).

- **The Concept: Functorial Semantics.** This describes mapping a "process" (your code execution) into a "vector space" while preserving the structure (composition).
- **Reference:** *Physics, Topology, Logic and Computation: A Rosetta Stone* (John Baez & Mike Stay).
- **Reference:** *Seven Sketches in Compositionality* (Brendan Fong & David Spivak).

- **Why check this:** It connects Physics (Flow/Energy) to Logic (Code). It validates your idea that a "Process" has a "Direction" and "Magnitude."

3. Neural-Symbolic AI

The exact hybrid you are building: merging "Fuzzy" Vectors (Neural) with "Strict" Logic (Symbolic/Pydantic).

- **The Concept: Vector Symbolic Architectures (VSA).** Representing structured data (trees, graphs) as high-dimensional vectors where operations like "binding" and "bundling" represent logical connections.
- **Reference:** *Neuro-Symbolic Artificial Intelligence: The State of the Art* (Besold et al.).
- **Why check this:** It offers techniques for encoding "Container A is inside Container B" purely using vector math, without recursion.

4. Representation Learning (Latent Space Traversal)

The origin of "Vector Arithmetic" (e.g., King - Man + Woman = Queen).

- **The Concept: Translation Vectors.** The idea that a logical operation (e.g., "Make Plural" or "Convert to JSON") corresponds to a specific *direction* vector in the latent space.
- **Reference:** *Efficient Estimation of Word Representations in Vector Space* (Mikolov et al. - The "Word2Vec" paper).
- **Why check this:** This is the "Ground" you asked for. The "Value" of your edge is the *Translation Vector* that moves data from State A to State B.

5. PIKE-RAG (Your Context)

Since you uploaded this, it is your specific architectural guide.

- **Reference:** *PIKE-RAG: sPeclalized Knowledge and Rationale Augmented Generation* (Wang et al., 2025).
- **Why check this:** Focus on their section on **"Knowledge Atomizing"**—this is essentially creating the "Basis Vectors" for your domain space.

Summary for your Investigation

Start by searching for **"Functorial Semantics"** and **"Graph Embedding Methods."** These two terms cover the mathematical validity of mapping your Pydantic processing chains to GPU-computable vectors.

What pydantic method did you mention to dynamicalle create class

The method is `create_model`.

It is a factory function provided directly by Pydantic that allows you to construct a `BaseModel` class at runtime using a dictionary of field definitions. This is the **Code-Side implementation of your "Functor"**—it takes the "Abstract" schema descriptions from your Blueprint and converts them into "Concrete" Python classes that your DeepAgent can actually use for validation.

How to use it in your Collider Factory

You import it from the main package: `from pydantic import create_model`

Here is how you use it to "hydrate" a node from your **Blueprint** (where you only have string/JSON descriptions) into a real class:

Dynamic Model Factory

Jan 14, 3:07 PM

Why this is critical for your Architecture

1. **The Bridge:** This function is the literal bridge between your **JSON Schema** (stored in `IOType`) and **Python Objects**.
2. **Agent Tools:** When your DeepAgent needs to use a tool found via vector search, you use `create_model` to generate the input arguments model for that tool on the fly, so the Agent (and `pydantic-ai`) knows exactly how to format the call.
3. **No Restarting:** You can inject new "Knowledge Atoms" (Definitions) into your running system without restarting the Python process to define new classes.

Cool we need that. Now the functorial semantics. For ex suppose we have python script, tools. With neat pydantics. Here our first defintion with zero graph node edge intelligence. This atomic definition can be combined in a container with othersnow here

is when the graph intelligence enters and is added to the first definitions. The dependant containers can be defined when the graph inside is readya new defintion instantiated. I called this mevhanism composite definition mechanism. I want to know if there is methods to retrive this "composite definition?

Yes, there is a very specific method to "retrieve" (or rather, **derive**) this Composite Definition.

In Computer Science and Category Theory, this method is called **Boundary Inference** or **Signature Derivation.**

You are effectively asking: *"How do I calculate the Pydantic Contract of the Black Box, given the Pydantic Contracts of the things inside it?"*

Here is the logic, the math, and the Python implementation.

1. The Logic: "The Unsatisfied Needs"

To retrieve a Composite Definition from a graph, you don't just sum up the parts. You must calculate the **Net Flow.**

- **Composite Input:** The set of all inputs required by internal nodes that are **NOT** satisfied by internal edges. (i.e., "What does the graph need from the *outside*?").
- **Composite Output:** The set of all outputs produced by internal nodes that are designated as "Final" (or are simply not consumed by anything else).

The Functorial View: If your atomic definitions are arrows f, g, h, and your graph wires them as $f \to g \to h$, the "Composite Definition" you retrieve is the single arrow $H = h \circ g \circ f$.

- **Domain (Input) of H:** Domain of f.
- **Codomain (Output) of H:** Codomain of h.

2. The Method: Boundary Schema Derivation

Here is the algorithm to "retrieve" this definition programmatically using Pydantic.

Composite Derivation

Jan 14, 3:15 PM

3. Scientific & Tech Sources

If you want to investigate the formal theory behind "retrieving the whole from the connected parts," look into these three specific domains.

A. Operad Theory (The Algebra of Wiring)

This is the heavy math version. An **Operad** is a mathematical structure that describes how to combine "little boxes" into "big boxes."

- **Concept: "Algebras over Operads."** It proves that if you know the inputs/outputs of the little boxes and the wiring diagram, the input/output of the big box is mathematically determined.
- **Search for:** *Operads of Wiring Diagrams* (Spivak, 2013).

B. Data Flow Analysis (Reaching Definitions)

This is the Compiler Theory version.

- **Concept: "Reaching Definitions"** and **"Live Variable Analysis."** Compilers use this to determine which variables are "live" at the entry of a block (the Inputs) and which are "live" at the exit (the Outputs).
- **Relevance:** Your graph is a Control Flow Graph (CFG). You are performing Live Variable Analysis to find the interface.

C. Component-Based Software Engineering (CBSE)

The architectural version.

- **Concept: "Interface Synthesis."** The automatic generation of a component's interface based on its internal composition.
- **Pattern: The Composite Pattern** (GoF), but specifically the "Transparent" variation where the Composite component exposes an interface derived from its children.

Summary

To "retrieve" the Composite Definition, you do not query it from a database (because it doesn't strictly exist until the graph is wired). You **synthesize** it by calculating the **Boundary Surface** of the graph:

Composite Definition = (Sum of All Inputs) - (Sum of Internal Wires)

This is the exact "Functorial" calculation that allows your recursive containers to work.

So where connections are made with edges between nodes that are dependanc8es of different containers

This is the specific architectural challenge of **Scope Crossing** or **Encapsulation**.

If you have **Container A** and **Container B**, and they are dependencies of a larger **Container Parent**, you cannot simply run a wire from a node deep inside A to a node deep inside B. That creates "Spaghetti Wiring" (tight coupling) and breaks your ability to reuse A or B later.

Instead, you use **Port Promotion** (also known as Pin Exporting).

Here is how the connection works mechanically, logically, and in your "Flat Graph" database.

1. The Mechanic: The "4-Step Handshake"

Data does not teleport from `Node_Inside_A` to `Node_Inside_B`. It must travel up to the boundary, cross the bridge, and travel down.

The Path of a Single Pydantic Value:

1. **Promotion (Up):** An internal node in Container A finishes. Its result is mapped to the **Output Boundary** of Container A.
2. **The Edge (Across):** The `Link` object in the Parent Container takes the value from `A.Output` and moves it to `B.Input`.
3. **Demotion (Down):** The value enters Container B's **Input Boundary**.
4. **Injection (In):** The value is mapped from the boundary to the specific **Internal Node** in B that needs it.

2. The Logic: Composite Nodes

In the eyes of the **Parent Container**, Container A and Container B are **not** graphs. They are **Black Boxes** (Atomic Nodes).

- **Parent's View:** "I have Node A and Node B. I connect A to B."
- **A's View:** "I am a graph. I have an internal node that produces a value, and I expose it to the world."
- **B's View:** "I am a graph. I require an input from the world to start my internal node."

This is where your `derive_composite_definition` function from the previous step is critical. It calculates the "Black Box" skin that allows A and B to sit next to each other in the Parent graph.

3. The Implementation (Pydantic + NetworkX)

Here is how you model this "Dependency Connection" in your Python code.

Wiring Logic

Jan 14, 3:38 PM

4. How to Store This (The "Flat" Solution)

Since we agreed to use a **Flat Graph** (Adjacency List) for database efficiency, here is how you distinguish "Internal Wires" from "Container Links."

You add a `scope_id` (or `context_id`) to your Edge table.

Edge ID	Scope ID (Parent)	Source Node	Target Node	Type
E1	`Container_A`	`Internal_Node_1`	`Internal_Node_2`	`Internal`
E2	`Container_A`	`Internal_Node_2`	`SELF (Output)`	`Promotion`
E3	`Main_Workflow`	`Container_A`	`Container_B`	`Dependency`
E4	`Container_B`	`SELF (Input)`	`Internal_Node_99`	`Injection`

The Query:

- When running **Container A**: You query edges where `Scope ID == Container_A`.
- When running **Main Workflow**: You query edges where `Scope ID == Main_Workflow`.

5. Scientific/Tech Context

- **VHDL / Verilog (Hardware Design):** In chip design, this is exactly how `PORT MAP` works. You define an entity (Container) with ports, and then you wire those ports in a higher-level architecture.
- **React (Frontend):** This is **Props**. You pass data into a Component (Container B) via props (Input Boundary), and the component passes data out via Callback/State (Output Boundary). You never reach directly into a Child Component's DOM.
- **Microservices (Orchestration):** In Kubernetes/Docker Compose, this is **Service Discovery**. Container A exposes port 80. Container B talks to `http://container-a:80`. They don't know about each other's internal processes, only the exposed interface.

Conclusion: The connection between dependencies is made at the **Parent Scope Level**, treating the child containers as single nodes. The data transmission relies on **Promotion** (Internal->Boundary) and **Injection** (Boundary->Internal) maps.